

LAYER 2 OF 13

APIS BACKEND LOGIC

How Your App Talks to Itself Behind the Scenes

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 2: APIs & Backend Logic—how your app talks to itself behind the scenes. When a user taps "Place Order" in your app, something has to happen between that tap and the order actually showing up in your system. That something is the backend. It's the invisible machinery that receives requests, runs business logic, talks to the database, and sends answers back to the screen.

You're not going to write server code by hand. You're going to describe what your app needs to do—"when someone submits this form, save their info and send them a confirmation"—and let your AI coding tool build the backend for you. Your job is to understand what's supposed to happen, check that it actually works, and catch mistakes before your users do. That's what this certification proves you can do.

This kit walks you through three levels of mastery. Tier 1 is for someone shipping their first app and needing the backend to just work. Tier 2 is for someone whose app is growing and needs reliable, organized backend systems. Tier 3 is for someone running a platform at scale where backend failures cost real money. Start at Tier 1. You can always come back for the rest.

Why It Matters

If your frontend is the face of your app, the backend is the brain. When it breaks, nothing works—forms don't submit, data doesn't save, payments don't process. And here's the painful part: your users see a blank screen or a spinning wheel, and they blame your entire product. A broken backend is invisible until it ruins someone's experience.

Here's the thing about AI coding tools: they're great at building backend endpoints and connecting things together. But they consistently make choices you didn't ask about—like not handling what happens when a request fails, or not validating the data that comes in, or creating an endpoint that returns way too much information to anyone who asks. AI builds exactly what you describe. If you don't know what to check for, you get gaps that turn into outages. This kit teaches you what to check for.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders shipping their first app

You understand how your app's screens talk to the server, can describe API endpoints to AI, and can verify they work correctly.

Tier 2

Growing teams with more complex workflows

You can manage API versioning, handle errors gracefully, implement webhooks, and keep your backend organized as it scales.

Tier 3

Platforms with high traffic and compliance needs

You can oversee API gateway architecture, rate limiting, microservices coordination, and backend monitoring at enterprise scale.

TIER 1 CERTIFICATION GOAL

You can describe what your app needs to do behind the scenes, use an AI coding tool to build the backend logic, test that it actually works, and ship an app where the frontend and backend talk to each other correctly.

What You Need to Know

You don't need to memorize how servers work. You need to understand a handful of concepts so you can describe what you want clearly and catch problems in what AI gives you back.

How requests and responses work: Every time your app needs data or needs to save something, it sends a request to the server, and the server sends back a response. Think of it like ordering at a counter—you ask for something (the request), someone in the back makes it, and they hand it to you (the response). If you don't understand this loop, you can't tell AI what to build.

What an API endpoint is: An API endpoint is a specific address on your server that does one thing. For example, `/api/orders` might be the address that handles everything about orders—creating them, listing them, updating them. When you tell AI "create an endpoint for managing orders," this is what gets built. Think of endpoints as doors into your app's backend—each one leads to a specific room.

REST basics (the most common pattern): REST is a set of rules for how your frontend talks to your backend. It uses simple actions: GET means "give me data," POST means "save new data," PUT means "update existing data," DELETE means "remove this." Your AI tool will build REST APIs by default. You need to know these four words so you can check that AI used the right one.

What happens when things go wrong: Your backend needs to handle errors—what happens when someone sends bad data, or the database is down, or a user asks for something that doesn't exist? AI often builds the "happy path" (everything works perfectly) and skips the error handling. You need to check for this every time.

The vibecoder workflow for backends: Describe what should happen when a user takes an action → AI builds the endpoint → you test it by sending a request and checking the response → you tell AI what to fix → repeat until it works → ship it. Same loop as frontend, but you're checking behavior instead of appearance.

Your Toolkit

Your AI coding tool builds the backend. These tools help you test and verify it.

AI coding tool (pick one): Cursor, Lovable, Bolt, Claude Code, Windsurf, or whatever AI-assisted builder you prefer. This is where you'll describe your backend logic and iterate on the output.

An API testing tool: Postman or Thunder Client lets you send requests to your API and see exactly what comes back—without using your app's frontend. Think of it as a way to talk directly to your backend and check its answers.

Your browser's network tab: Chrome DevTools has a Network tab that shows every request your app makes behind the scenes. When something isn't working, this is where you look first—it shows you exactly what was sent and what came back.

Certification Exam Topics

Every exam question is scenario-based. You'll see a situation and need to identify what's right, what's wrong, or what to do next. Here's what gets tested:

Request and response basics: Can you identify what type of request (GET, POST, PUT, DELETE) your app should use for a given action?

Endpoint design: Can you tell whether AI organized your API endpoints in a clean, logical way—or created a confusing mess of URLs?

Error handling: Can you spot when AI built an endpoint that only works when everything goes perfectly and breaks silently when something goes wrong?

Data validation: Can you identify when an endpoint accepts any data without checking it first—like saving an order with no price or a user with no email?

Status codes: Do you know what it means when your API returns a 200 (success), 404 (not found), or 500 (server broke)—and can you check that AI uses the right ones?

Request/response format: Can you read a JSON response from your API and verify it contains the right data in the right structure?

Server-side logic: Can you describe a business rule (like "don't let users order more than 10 items") and verify AI implemented it correctly?

Your AI workflow: Can you describe a backend feature clearly, test the result with an API tool, give AI useful feedback when the output is wrong, and iterate to a working result?

Common Pitfalls

These are the mistakes vibecoders make most often at this layer. No judgment—they're easy to make. But if you recognize any of them in your own workflow, fix them before sitting for the exam.

Only testing the happy path. Your endpoint works when you send perfect data. What about empty fields, missing values, or someone sending the wrong data type? AI almost never builds error handling unless you specifically ask for it.

Not checking what your API actually returns. AI built the endpoint, you see it "works," but you never looked at the actual response data. It might be returning sensitive information like passwords, internal IDs, or data the user shouldn't see.

Skipping authentication on endpoints that need it. AI creates an endpoint for deleting accounts but doesn't check if the person making the request is actually allowed to do that. Always verify that protected actions require a logged-in, authorized user.

Building everything as one giant endpoint instead of separate, focused ones. If your `/api/do-everything` endpoint handles orders, users, and payments, you have a maintenance nightmare. Each endpoint should do one clear thing.

Not thinking about what happens when your server is slow or down. Your frontend shows a loading spinner forever, or worse, silently fails. Every request needs a timeout and a clear "something went wrong" message.

Treating AI's first backend output as production-ready. Backend code has the same first-draft problem as frontend code—it works for the demo but breaks under real conditions. Always test with realistic data and edge cases.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Can you describe a backend feature to AI clearly enough that you get a working endpoint on the first or second try?

Have you tested your API endpoints with an API testing tool (not just through your app's frontend)?

Can you read a JSON response and verify it contains the right data in the right structure?

Do your endpoints return helpful error messages when something goes wrong—not just a blank 500 error?

Have you checked that your endpoints validate incoming data before processing it?

Can you open your browser's network tab and trace a request from button click to server response?

Do you know the difference between GET, POST, PUT, and DELETE—and can you tell when AI used the wrong one?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check on your backend. It checks the same things the certification exam covers.

Review my app's backend API and check the following. For each one, tell me pass or fail with a specific example:

Endpoint organization: Are my API endpoints logically organized with consistent naming and proper HTTP methods (GET, POST, PUT, DELETE)?

Error handling: Does every endpoint handle errors gracefully—returning helpful error messages with correct status codes instead of crashing silently?

Input validation: Does every endpoint check incoming data before processing it—rejecting missing fields, wrong data types, and invalid values?

Authentication: Are protected endpoints properly checking that the user is logged in and authorized before allowing the action?

Response quality: Are responses returning only the data the frontend needs—not leaking sensitive fields like passwords, internal IDs, or private user data?

Performance: Are there any endpoints that will be slow under load—missing pagination, loading too much data at once, or making unnecessary database calls?

Give me an overall score out of 6 and list the top 3 things to fix first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 2 certification exam at your target tier.

The best way to prepare: build something real. Pick a project that needs a backend—even a simple one like a contact form that saves to a database—and go through the full loop. Describe it to AI. Test the endpoints. Check error handling. Fix the gaps. Ship it. Every mistake you catch during building is exactly what the exam tests.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.